SHENZHEN UNIVERSITY

COURSE

International Marketing Strategy
Shenzhen University

Data Analytics Module, Lecture 5 — Introduction to Machine Learning
Supervised, unsupervised, deep and reinforcement learning

Lecturer: Xieer Dai (David)

Shenzhen University • Weeks 3–14 · Thursday, periods 11–13

Course Code 0205360801 | Weeks 1–18 · Wed 14:15–16:45 · Thu 10:15–11:45

Venue: Sifang Building South 207 | Office hours: Thu 16:00–17:00 | E-mail:

What Machine Learning Is

Mitchell's definition (1997)

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

Read it as three slots you must fill before any modelling starts:

- **T** — what decision the model supports;
- **P** — how you will know it is any good;
- **E** — what data it is allowed to learn from.

Source: Tom M. Mitchell, *Machine Learning*, McGraw-Hill, 1997.

The one-line version

Find the pattern,
then apply the pattern.

No human writes the rule. The rule is estimated from examples — and it is only as good as the examples and the measure you chose.

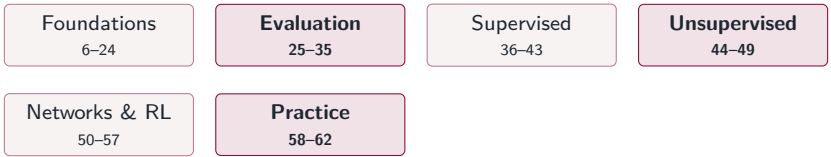
Learning Objectives

After this lecture you should be able to:

1. distinguish supervised, unsupervised, deep and reinforcement learning, and say which problem each one fits;
2. explain generalisation, overfitting and underfitting, and describe the bias–variance trade-off behind them;
3. describe the main supervised learners — penalised regression, SVM, k-NN, CART, ensembles, random forest, neural networks — and match each to a problem;
4. describe the main unsupervised methods — PCA, k-means, hierarchical clustering, anomaly detection — and match each to a problem;
5. explain what a neural network computes, what depth adds, and how reinforcement learning differs from supervised learning;
6. read an ML result critically: leakage, hold-out discipline, class imbalance, interpretability, and the difference between prediction and causality.

Source: Structure follows the module reading (Introduction to Machine Learning); examples are re-cast for international marketing decisions.

Roadmap



Spine: what learning is → how you know it worked → the algorithm families → networks and reinforcement → how to use it responsibly

One promise: by the end you can look at a marketing problem and say which family of methods fits, what evidence would convince you, and what the model cannot tell you.

Where L5 Sits



What you already hold

a clean analysis table, a data dictionary, descriptive statistics and charts — and a question worth answering.

What L5 adds

a prediction machine and, more importantly, the discipline for deciding whether its predictions can be trusted.

Source: In the project, L5 is where you justify the model behind a forecast — demand, churn, response or price — rather than reporting a correlation.

Learning from Examples, Not Rules

The classical program

A human writes the rule:

$$\text{spam} \;=\; (\text{“free”} \wedge \text{link}) \;\vee\; \ldots$$

Every exception must be anticipated, and the rule set is only as good as the person who wrote it.

The learning program

A human supplies examples with answers and a loss; the machine searches for the rule:

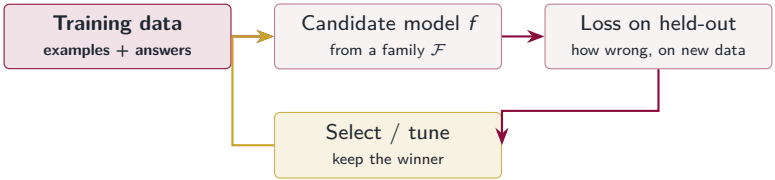
$$f^{*} = \arg\min_{f \in \mathcal{F}} \; \frac{1}{n} \sum_{i=1}^{n} L\big(y_i, f(x_i)\big)$$

Exceptions are absorbed by the data rather than enumerated by hand.

This is why machine learning is described as **extracting knowledge from data**: the emphasis is on producing structure or predictions without a human specifying the mechanism, and on generalising from known cases to new ones.

Source: The trade-off is real: a learned rule is cheaper to obtain but harder to explain, audit and defend than a written rule — see slide 58.

The Pattern Loop



What is being searched

the family (linear? tree? network?), the features, and the tuning parameters that control flexibility.

What stops the loop

not the training error — that always falls. The loop stops when error on data the model has never seen stops improving.

Why Now: Data, Compute, Algorithms

Data

Platforms, logs, text, images and payments turned small, designed samples into large, observed populations. Deep models need the latter.

Compute

GPUs and cheap cloud training made it feasible to fit models with millions of parameters — and to re-fit them weekly.

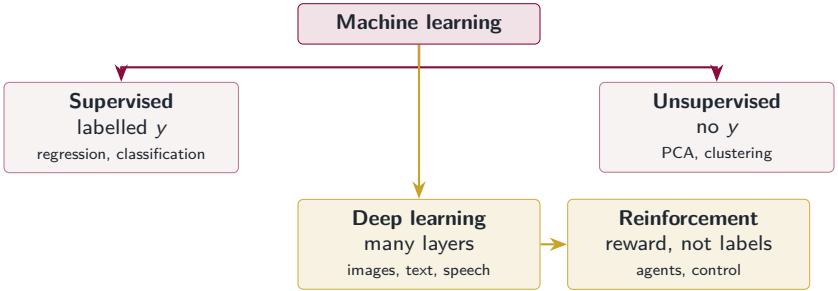
Algorithms

Better regularisation, optimisation and architectures (trees $\rightarrow$ ensembles $\rightarrow$ networks) turned raw compute into accuracy.

The marketing consequence: the binding constraint moved from can we estimate it? to do we have the right target, the right sample and the right validation?

Source: The three-column framing follows the standard account of the deep-learning take-off; see also L1 on the AI landscape.

The ML Family Tree



Supervised learns a mapping from inputs to a known answer.

Unsupervised looks for structure when no answer is recorded.

Deep and **reinforcement** are defined by architecture and by feedback, not by the presence of labels.

Supervised Learning: Two Tasks

Regression — predict a number

$$\hat{y} = E[Y \mid X = x]$$

Demand next quarter, price premium, customer lifetime value, expected response rate.

Classification — predict a class

$$\hat{p}(y) = \Pr(Y = y \mid X = x)$$

Fraud or not, churn or stay, which of five segments, which product will be chosen.

Both need labelled observations: for each row, the answer must already be recorded. That requirement is the practical bottleneck in marketing data — and the reason so much work goes into constructing labels (did the customer actually churn? did the campaign actually cause the sale?).

Source: Independent observations and a stable environment are assumed; when the environment shifts, the mapping you learned stops holding — the theme of slide 18.

Unsupervised Learning: Two Tasks

Dimension reduction

Reduce the number of features while retaining the variation that carries the information — so that a table with 300 columns becomes a map with 2 axes.

Clustering

Group observations so that members of a group are more like each other than like outsiders — with the number of groups either given or discovered.

Why marketing cares

- reduction: perceptual maps, index construction, de-noising a survey before modelling;
- clustering: data-driven segments from behaviour and financials, rather than pre-set sector or country labels — the difference between "the German market" and "the four kinds of buyer who happen to live in Germany".

Deep Learning

Deep learning applies many-layer networks to tasks where the relevant features are themselves hard to hand-code: image classification, face recognition, speech, natural language.

What depth buys

Each layer builds features from the previous one: edges → shapes → parts → objects. You supply raw pixels or tokens, not "number of edges".

What it costs

More data, more compute, less transparency; and a strong tendency to fit artefacts of the training set unless validation is strict.

Deep learning is not a separate learning paradigm: a deep network trained on labelled data is supervised learning; one trained with a reward signal is reinforcement learning.

Source: Lecture 6 (AI Foundation) covers how these networks are built and evaluated in more depth.

Reinforcement Learning



The idea

Learn a policy that maximises cumulative reward over time, by acting and observing consequences — not by copying labelled answers.

Marketing uses

bid and budget allocation, sequential pricing, personalised sequencing of messages, inventory and delivery decisions.

The hard part is not the algorithm but the experiment: you need a simulator or a controlled live test, because the only way to learn is to try actions that may be temporarily worse.

Source: AlphaGo (2017) is the canonical demonstration; business applications are usually bandits or budget-allocation problems rather than games.

ML in Finance: Seven Applications

- **Process automation** — straight-through handling of documents and requests;
- **Document analysis** — extracting terms, covenants and risk language from filings;
- **Portfolio management** — return forecasting, risk factors, rebalancing;
- **Algorithmic trading** — execution timing, microstructure signals;
- **Digital assistants** — answering client and staff questions from internal knowledge;
- **Risk management** — credit scoring, stress testing, early-warning signals;
- **Fraud and money laundering** — anomaly detection over transaction graphs.

Note the pattern: in every case the model is one component of a decision process with a cost of error — it never stands alone.

Source: The seven-way list follows the investment-industry reading used in the module; the marketing equivalents are on the next slide.

ML in Marketing: Seven Applications

- **Recommendation** — which product, content or offer for this user;
- **Demand forecasting** — volume by SKU, market and channel;
- **Churn and retention** — who is about to leave, and what would keep them;
- **Pricing and promotion** — elasticity, discount depth, markdown timing;
- **Creative and copy testing** — predicting response before spending;
- **Sentiment and review mining** — turning text into measurable attributes;
- **Campaign optimisation** — budget allocation, bidding, audience selection.

Two of these are classification (churn, response), two are regression (demand, elasticity), and three mix prediction with optimisation — which is where reinforcement learning re-enters.

Three Worked Examples

1. **Fraud detection.** Label: transaction later confirmed fraudulent. Features: amount, merchant category, time, device, geography. A classification problem with severe class imbalance — 0.1% positives — so accuracy is useless and precision/recall at a fixed review budget is the real criterion.
2. **Support chatbot.** Learns from past conversations which answer resolved a question. The label is noisy ("resolved" is a proxy), and the cost of a wrong answer is asymmetric.
3. **Filing text as a signal.** Year-on-year changes in 10-K/10-Q language — especially negative shifts in management discussion and risk sections — carry information about subsequent equity returns. Here the model is text $\rightarrow$ feature $\rightarrow$ prediction, and the hard part is showing the signal is not just known risk factors.

Source: The filing-language example follows the accounting-and-finance text-analysis literature; the point for us is the label and the validation, not the specific result.

ML versus Econometrics

	Prediction (ML)	Causal inference (econometrics)
Question	what will happen to y?	what happens to y *if we intervene*?
Data regime	often $p > n$; many features, few rows	usually $n \gg p$; few, chosen regressors
Selection	data-driven, by out-of-sample error	theory-driven, by identification
Criterion	prediction error on held-out data	unbiasedness of a parameter
Failure mode	overfitting, leakage, drift	omitted variables, wrong counterfactual
Output	a forecast or a score	an estimated effect with a standard error

Neither column is "better". The mistake is to answer a causal question with a predictive model, or to demand a structural interpretation from a model built only to forecast.

Source: The contrast is standard in the economics-and-ML literature (Mullainathan–Spiess, Athey); L12 returns to identification in detail.

Prediction in a Stable Environment

The recipe

1. define a family of models and a grid of tuning parameters;
2. split the data into k folds (commonly 10);
3. fit on $k-1$ folds, score on the held-out fold;
4. repeat, average — pick the parameter with the lowest mean squared error;
5. refit the winner on all data.



The reason ML works here is uncomfortable but important: you can evaluate performance without additional assumptions. You do not need the model to be "true" — only to predict better on data it has not seen.

Source: This is the logic of `cross_validation` in scikit-learn and of `vfold` in Stata; both are used in the course project.

What We Say v. What We Do: ML

What we say

- ML *is* data science and statistics — is there anything else?
- we speak the language of solving problems: advertising allocation, salesperson prioritisation;
- the analyst does not have to choose — the data chooses;
- all that matters is prediction.

What we do

- use predictive models and set aside causality, equilibrium and feedback;
- worry about interpretability, reliability, robustness and adaptability — yet have little lever to make the algorithm optimise for them;
- accept whatever the loss function encodes, including its blind spots.

The gap is not hypocrisy; it is a specification problem. "Accurate" and "useful" are different objectives, and only one of them is in the loss function.

Contrast with Traditional Econometrics

The classical regime: $n \gg p$

- in-sample MSE is a decent proxy for out-of-sample MSE;
- OLS is BLUE; if overfitting is not a threat, parsimony buys nothing;
- OLS uses all the data and minimises in-sample error;
- but it fails badly when $p \sim n$, and has no solution when $p > n$.

Why ML changes the answer

- ML methods are built for $p > n$: penalisation, selection, ensembles;
- regularisation trades a little bias for a large reduction in variance;
- the criterion is out-of-sample error, not fit;
- and the search over models is itself validated, not assumed.

Source: When the number of candidate regressors approaches the number of observations, the classical inference machinery stops being informative before it stops returning numbers.

What We Say v. What We Do: Econometrics

What we say

- causal inference and counterfactuals;
- "the model was given to us";
- we report estimated effects with appropriate standard errors;
- plus a few additional specifications for robustness.

What we do

- run OLS or IV, try many functional forms;
- report standard errors as if only one model had been run;
- have assistants run hundreds of regressions and pick "representative" ones;
- build complex structural models on assumptions we cannot test well.

Read together with the previous slide, the lesson is symmetric: both traditions have a research-design problem, and each can borrow the other's discipline.

Key Lessons for Econometrics

Borrow the predictive parts

- decompose a problem into predictive and causal components;
- use off-the-shelf ML for the predictive component;
- select models with data, but tailor the criterion to the economic goal;
- keep the ability to do inference on the parameters that matter.

Borrow the robustness habits

- ML always keeps a test set — so should we;
- evaluate on unusual periods (crisis days, price changes), not only average days;
- exploit natural experiments: A/B tests, algorithm changes, policy shifts.

Computational tools that make this practical: stochastic gradient descent for large models, and variational inference for Bayesian ones.

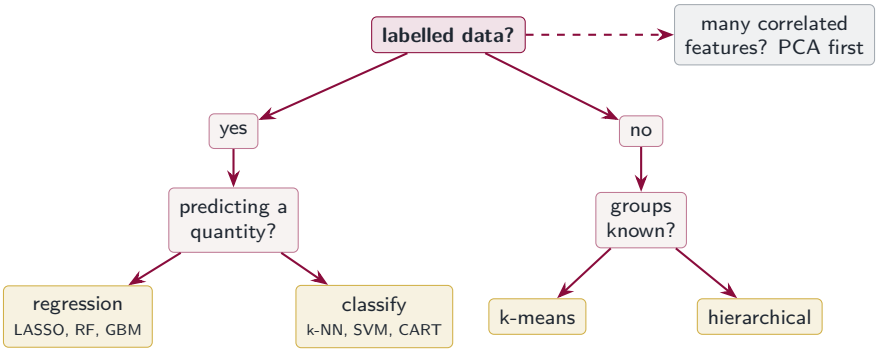
Source: Mullainathan & Spiess (JEP 2017); Athey (Science 2017); Varian (2014) for the applied-econometrics view.

Choosing an Algorithm: Five Rules

1. **Many correlated features?** Reduce dimension first — PCA — before modelling.
2. **Prediction or classification?** If prediction, split again by shape: penalised regression / LASSO for linear structure; CART, random forest, neural networks for non-linear structure.
3. **Classification?** Split by whether the data are labelled: supervised classifiers if yes, clustering if no.
4. **Labelled and classification:** k-NN and SVM for linear boundaries; CART, random forest, networks for non-linear ones.
5. **Unlabelled and clustering:** k-means when the number of groups is known; hierarchical when it is not; networks when the structure is strongly non-linear.

Source: This decision list is the compressed form of the module reading's selection guidance; the full map is on the next slide.

Estimator Selection Map



Practical advice: start with the simplest model that could work and a proper hold-out. If a LASSO or a small tree is within a few points of a network, ship the simpler one — it will fail more predictably.

Source: Compare the scikit-learn estimator map (`https://scikit-learn.org/stable/tutorial/machine_learning_map/`), which adds the sample-size branches.

Generalisation

Generalisation is the ability of a fitted model to retain its explanatory power on data it has never seen. Everything else in this section follows from taking that sentence literally.

A model that generalises

captures the stable relationship between inputs and outcome; its error on new data is close to its error on the data used to fit it.

A model that does not

has memorised the training rows, including their noise; its error on new data is much worse — sometimes worse than a naive baseline.

Hence the rule that governs the rest of the lecture: evaluate on new data, never on the fitting data. The training error is a description of the past, not a forecast.

Source: Formally, generalisation error is the expected loss over the joint distribution $P(X, Y)$; we estimate it with a held-out sample.

Overfitting

An overfit model has absorbed the noise and random fluctuations of the training sample into the relationship it learned. Those fluctuations do not exist in new data, so the model's predictions inherit an error that is not there to be found.

Symptoms

- training error much lower than validation error;
- performance collapses when the sample is refreshed;
- small changes in the input produce large swings in the output;
- the model needs the exact features it was trained on.

Why it happens

- too many parameters relative to rows;
- features that encode the answer (leakage);
- tuning on the test set, repeatedly;
- selection among hundreds of models without correction.

Source: The practical test is simple: if validation error is not close to training error, treat the model as unproven.

Underfitting

Underfitting is the opposite failure: the model is too simple to capture the relationship, so it does badly on both training and new data.

Typical causes

- the functional form cannot represent the pattern (a line for a threshold effect);
- important features are missing or badly measured;
- regularisation is too strong;
- too little training data, or too few iterations.

What it looks like

- high error everywhere, and flat learning curves;
- residuals with visible structure — the model misses a real pattern;
- adding features or flexibility improves both errors.

Underfitting is easier to diagnose and harder to admit: it often means the data cannot answer the question as posed.

Bias, Variance and Base Error

Expected squared prediction error decomposes into three parts:

$$\mathbb{E}\big[(y-\hat{f}(x))^2\big] = \underbrace{\sigma^2}_{\text{irreducible}} + \underbrace{\big(\text{Bias}[\hat{f}(x)]\big)^2}_{\text{too simple}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{too unstable}}$$

Bias

how far the average model is from the truth. Wrong assumptions → high bias → underfitting, high in-sample error.

Variance

how much the fitted model moves when the data change. Sensitivity to noise → high variance → overfitting, high out-of-sample error.

Base error

σ^2: randomness in the outcome that no model can remove. It sets the floor for any comparison.

Source: This is why "reduce the error" is not a goal by itself: you can only trade bias against variance, never eliminate both.

Partitioning the Data

Three non-overlapping samples

- **Training** — fit the model ("in-sample");
- **Validation** — compare candidates and tune ("out-of-sample");
- **Test** — touch once, to confirm the final model ("out-of-sample").

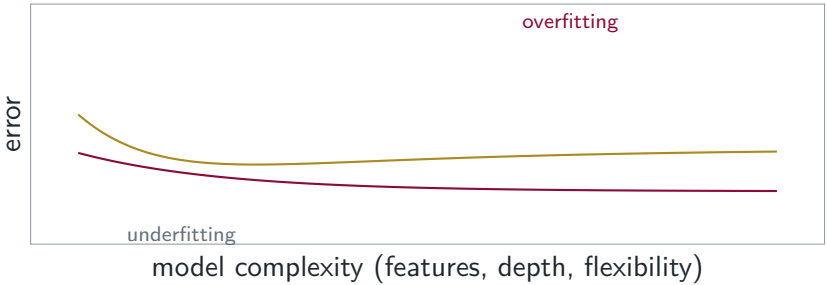


Common splits: 60/20/20 or 70/15/15. With few rows, use cross-validation inside the training block instead of a fixed validation set.

The test set is a budget, not a resource. Every look at it spends it; after a few looks it is just another validation set with a more flattering name.

Source: Partition before you look at the data — after the first plot, every choice you make is informed by the test rows.

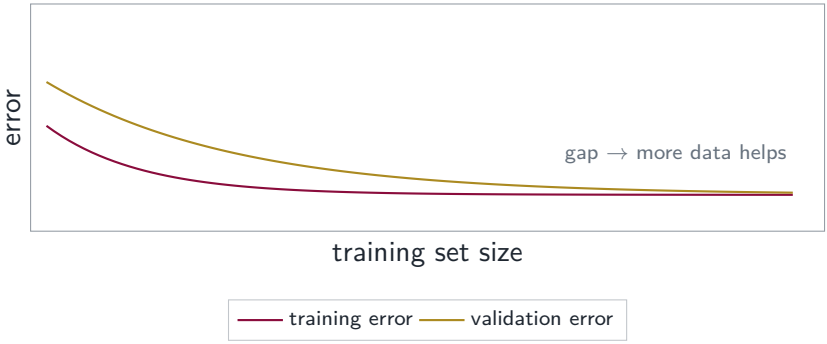
Complexity and the Error Curve



The shape is the whole story: training error falls monotonically with flexibility, validation error falls and then rises. The minimum of the gold curve — not the red one — is your model.

Source: Schematic, drawn to show the shape rather than measured values.

The Learning Curve



A learning curve plots error against the amount of training data. If the two curves converge at a high level, more data will not help — the model or the features are the limit. If a wide gap persists, more data (or less flexibility) will.

Source: The term is also used for experience curves in operations; here it is strictly a diagnostic for model capacity.

Two Principles for Preventing Overfitting

1) Limit complexity

Constrain the model during selection and training, and pay an explicit overfitting penalty for flexibility: fewer features, shallower trees, stronger regularisation, early stopping.

2) Sample properly

Estimate out-of-sample error directly by holding data back — a validation set, or cross-validation when data are scarce.

Why both are needed

Complexity control without honest validation is guesswork; validation without complexity control just tells you, accurately, that you overfit. The two mechanisms work on different sides of the same trade-off.

Source: These are the two levers in the module reading: penalisation and data partitioning.

Occam's Razor and the Penalty

The first principle descends from **Occam's razor**: among explanations with equal fit, prefer the simpler. In supervised learning it becomes an explicit penalty in the objective:

$$\min_{\beta} \sum_{i=1}^{n} \left(y_i - x_i'\beta\right)^2 \; + \; \lambda \, \mathcal{P}(\beta)$$

where $\mathcal{P}(\beta) = \|\beta\|_2^2$ (ridge) or $\|\beta\|_1$ (LASSO), and λ is chosen by validation.

What the penalty does

- shrinks coefficients towards zero;
- LASSO sets some exactly to zero — automatic feature selection;
- larger $\lambda \to$ simpler model $\to$ more bias, less variance.

The honest version of "the simplest solution is usually right" is therefore not a slogan but a tuning parameter you must validate.

Source: Hoerl & Kennard (1970) for ridge; Tibshirani (1996) for the LASSO.

Sampling Bias and the Three-Way Split

The second principle is the discipline of the controlled experiment: samples must be representative, and comparisons must be made against a comparable group drawn from the same distribution.

Training sample	Validation sample	Test sample
labelled rows where Y is known; the model sees these.	used to compare candidate solutions and tune the chosen one.	held aside to confirm predictive or classifying power on genuinely new data.

Two sampling traps dominate marketing applications: temporal leakage (random splits put future rows in training) and entity leakage (the same customer appears in train and test). Split by time and by customer, not by row.

Source: In panel marketing data, a time-based split is almost always the honest one.

Cross-Validation in Practice

k-fold, step by step

1. shuffle (or, for time series, roll forward — never shuffle);
2. split into k folds, commonly 5 or 10;
3. for each candidate parameter value: fit k times, score on the omitted fold;
4. average the k scores; keep the best parameter;
5. refit on all training data and report the test score once.

Variants worth knowing

- **stratified**: preserve the class ratio in every fold (imbalanced churn);
- **grouped**: keep all rows of a customer in one fold;
- **time-series**: expanding-window splits only;
- **nested**: an inner loop for tuning, an outer loop for the estimate.

Cross-validation is not a formality: choosing the split is choosing what "generalise" means for your problem.

Supervised Learning Defined

Supervised learning fits a function from inputs to a recorded output, using examples where the answer is already known:

$$f\colon X \longrightarrow Y, \qquad \hat{f} = \arg\min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} L\big(y_i, f(x_i)\big)$$

What you must supply

a defensible Y (what actually happened), features available at decision time, and a loss that reflects the cost of being wrong.

What you get

a mapping that can be applied to unseen rows — with a measurable error, provided the unseen rows come from the same world.

The quality of the result is dominated by the label definition, not by the algorithm. Most failed marketing models are failed label designs.

The Supervised Family

Classical statistical learners

- **Linear regression** — continuous outcome, additive effects;
- **Logistic regression** — binary outcome, interpretable coefficients;
- **Naive Bayes** — text and high-dimensional sparse counts;
- **Linear discriminant analysis** — classification with Gaussian classes;
- **Penalised regression** — many correlated predictors.

Machine learners

- **k-nearest neighbours** — local similarity, no global form;
- **Support vector machines** — wide-margin boundaries, kernels;
- **CART** — trees of if–then rules;
- **Ensembles** — bagging, boosting, random forests;
- **Neural networks** — flexible non-linear mappings.

The boundary between the two columns is historical, not mathematical: penalised regression is a statistical estimator and a machine learner at the same time.

Linear and Logistic Regression as Baselines

Linear regression

$$y = \beta_0 + \sum_{j=1}^{p} \beta_j x_j + \varepsilon, \qquad \hat{\beta} = \arg\min \sum_i \left(y_i - x_i'\beta\right)^2$$

Coefficients are marginal effects — which is why economists like them and why they are so easy to over-interpret when features are correlated.

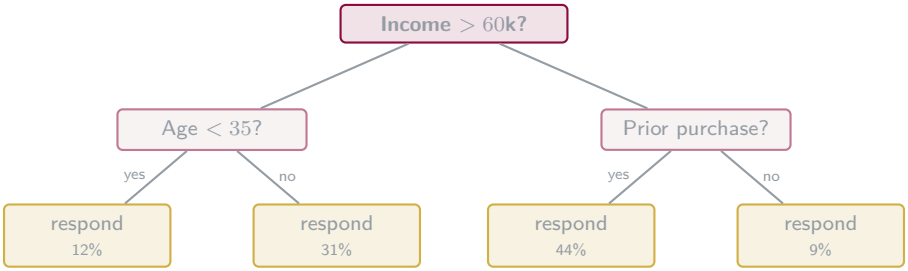
Logistic regression

$$\Pr(y = 1 \mid x) = \sigma(x'\beta) = \frac{1}{1 + e^{-x'\beta}}$$

Effects are on the log-odds scale; the marginal effect on the probability depends on where you evaluate it. Report predicted probabilities, not just odds ratios.

Always fit the baseline first. If a gradient-boosted forest cannot beat logistic regression by a meaningful margin on held-out data, the extra complexity is not paying for itself.

Classification and Regression Trees



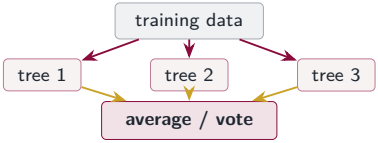
A tree partitions the feature space into rectangles and predicts a constant in each. Splits are chosen greedily to reduce impurity (Gini, entropy) or squared error.

Strength: interpretability and interactions for free. Weakness: unstable — a small change in the data can redraw the tree — and prone to overfitting unless pruned.

Ensembles and Random Forests

Bagging

Fit many trees on bootstrap samples and average. Averaging reduces variance without adding much bias — the single most reliable improvement in applied ML.



Random forest

Bagging plus a random subset of features at every split, which decorrelates the trees and makes the average work harder.

Boosting (gradient boosting, XGBoost, LightGBM) is the other family: trees are fitted *sequentially*, each on the previous model's errors. Usually more accurate than a forest, and easier to overfit if the learning rate and depth are not validated.

Source: Breiman (2001) for random forests; Friedman (2001) for gradient boosting.

Nearest Neighbours and Support Vector Machines

k-nearest neighbours

Predict from the k most similar training rows — no global model, only a distance. Strengths: instant, non-parametric, naturally multi-class. Weaknesses: sensitive to scaling, slow at prediction time, and useless in high dimensions without reduction.

Support vector machines

Find the boundary with the widest margin between classes; a kernel lets it bend. Strong when p is large and n moderate (text, genomics). Costs: kernel choice, tuning, and poor probability calibration by default.

Both are margin/similarity methods

They work because local geometry carries the signal. When features are noisy or unscaled, that geometry is meaningless — standardise first, and reduce dimension if p is large.

Choosing Among Supervised Learners

Method	Interpretable	Non-linear	Handles $p > n$	Best suited to
Linear / logistic	high	no	no	baseline, effect reporting
Penalised (ridge/LASSO)	high	no	yes	many correlated features, selection
k-**NN**	low	yes	poorly	small n, low p, similarity tasks
SVM	low	via kernel	yes	text, high p, medium n
CART	high	yes	yes	rules for operations, missing data
Random forest	medium	yes	yes	general tabular default
Boosting	low	yes	yes	accuracy-first tabular problems
Neural network	very low	yes	yes	images, text, very large n

Read the table as a starting point: the ranking that matters is the one your validation set produces on your data.

Reading a Classifier: Beyond Accuracy

With a 2% churn rate, a model that predicts "nobody churns" is 98% accurate and completely useless. Report the confusion matrix and the derived measures:

The four counts

	predicted yes	predicted no
actual yes	TP	FN
actual no	FP	TN

$$\text{precision} = \frac{TP}{TP+FP}, \quad \text{recall} = \frac{TP}{TP+FN}$$

What to report

- precision at the operating budget (how many calls can you make?);
- recall at that same point;
- the full ROC or precision–recall curve, not one number;
- calibration: are predicted probabilities close to observed rates?

Source: Choose the threshold from the cost of a false positive versus a false negative — a marketing decision, not a statistical one.

Unsupervised Learning Defined

Unsupervised learning finds structure in data without a recorded answer. Nothing tells the algorithm what is "right"; the criterion is internal (variance, distance, density).

Weak / semi-supervised

a small labelled subset plus a large unlabelled one — the usual situation when labels are expensive.

Self-supervised

the data label themselves (predict the next token, the masked word, the missing patch). The basis of modern language models.

Fully unsupervised

no labels at all: clustering, dimension reduction, density estimation, anomaly detection.

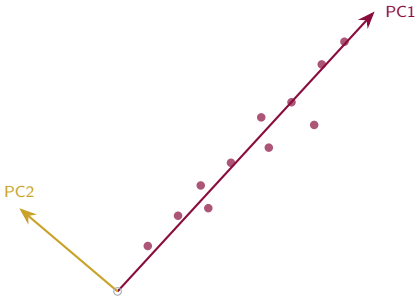
Because there is no ground truth, evaluation is judgement: a segmentation is useful if it predicts something you care about — response, margin, retention — not because the clusters look tidy.

Principal Components Analysis

PCA finds orthogonal directions that maximise variance, and re-expresses each row in those directions:

$$z_1 = w_1' x, \quad \text{with } w_1 = \arg\max_{\|w\|=1} \mathrm{Var}(w'x)$$

Keep the first k components and you have a lower-dimensional summary that retains as much variation as k dimensions allow.



Use it for perceptual maps, index construction, de-noising before clustering, and to make $p > n$ tractable. Do not interpret components as meaningful variables unless you check their loadings; "PC1" is a direction, not a concept.

Source: Always scale features before PCA — otherwise the variable with the largest units wins.

K-Means Clustering

The algorithm

1. choose k initial centroids;
2. assign each row to the nearest centroid;
3. recompute each centroid as the mean of its members;
4. repeat until assignments stop changing.

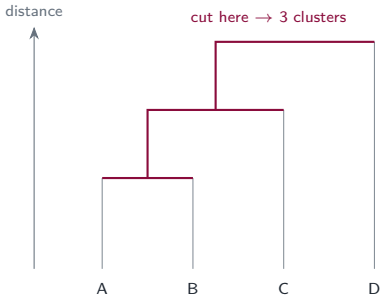
Choosing k

- elbow: plot within-cluster SS against k;
- silhouette: how much closer a point is to its own cluster than the next;
- business: does the segmentation support different actions?

It minimises within-cluster sum of squares — so it is sensitive to scaling, to outliers, and to the choice of k.

In marketing the last criterion dominates: a five-segment solution that no team can act on is worse than a three-segment solution with clear offers attached.

Hierarchical Clustering



Agglomerative clustering starts with every row as its own cluster and merges the closest pair until one cluster remains. The dendrogram records the whole merge history.

Strength: you do not fix k in advance — cut the tree where the distance jumps. Weakness: $O(n^2)$ memory, so it does not scale to millions of customers.

Anomaly Detection

The task

Flag observations that do not fit the learned pattern: fraudulent transactions, impossible prices, scraped listings, broken tracking.

Approaches

- density: isolation forest, local outlier factor;
- distance to the nearest cluster centroid;
- reconstruction error from PCA or an autoencoder;
- rules as a fallback, always.

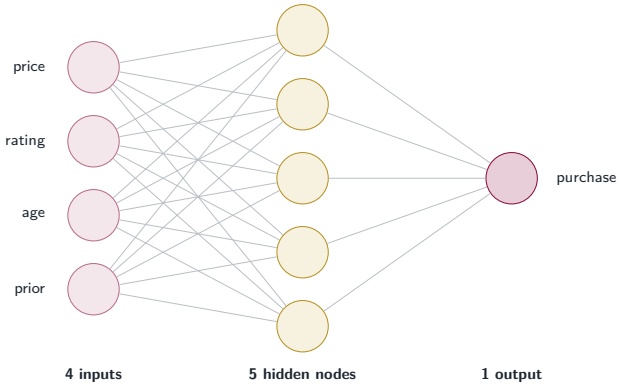
Two practical notes. First, anomalies are rare by construction, so precision at a small alert budget is the only meaningful metric. Second, anomalies are not errors: a genuine viral spike looks exactly like a tracking bug, and only a human can tell them apart.

Choosing Among Unsupervised Methods

Method	Needs k?	Scales to large n?	Best suited to
PCA	no	yes	correlated features, perceptual maps
k-**means**	yes	yes	large, roughly spherical segments
Hierarchical	no	no	small samples, exploratory taxonomy
Isolation forest	no	yes	fraud, defects, data-quality flags
Gaussian mixture	yes	yes	soft assignment, overlapping segments
Autoencoder	no	yes	high-dimensional structure, reconstruction

In all of these, the deliverable is not the cluster plot — it is a table of segment profiles with sizes, value, behaviour and a proposed action.

Anatomy of a Neural Network



The numbers 4, 5, 1 are **hyperparameters**: they define the architecture, not the fitted weights. A network is just a flexible function with many parameters — and every one of them has to be estimated from data.

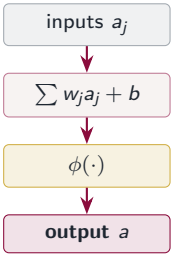
Source: This is the exhibit used in the module reading (input layer → one hidden layer → output node).

Inside a Node

Each node does two things, in order:

$$z = \sum_{j=1}^{m} w_j a_j + b, \qquad a = \phi(z)$$

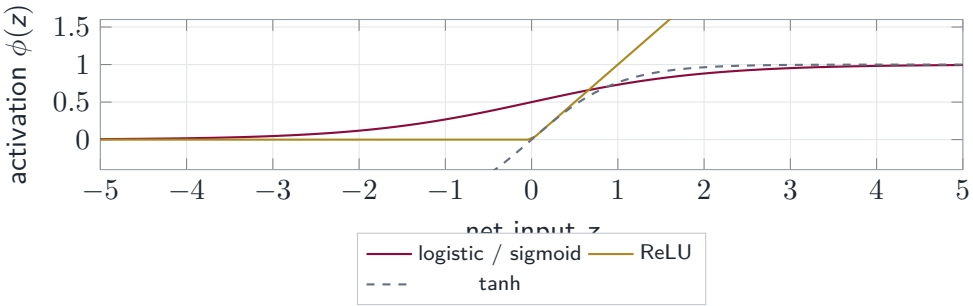
The **summation operator** forms the weighted net input; the **activation function** ϕ turns it into the node's output. Stacking these two operations is what makes a network non-linear.



Reading the parameters

w_j = how much input j matters; b = the node's baseline propensity to fire; ϕ = the shape of the response.

Activation Functions



Sigmoid squashes to $(0, 1)$ and reads like a probability; ReLU is cheap and avoids saturation; tanh is centred at zero. Non-linearity is the point: it lets the rate of change differ at different input levels — which a single weighted sum cannot do.

Forward Propagation

Prediction runs forward through the layers: inputs → hidden → output, with each layer applying its own weights and activation.

$$a^{(1)} = \phi\big(W^{(1)}x + b^{(1)}\big), \qquad a^{(2)} = \phi\big(W^{(2)}a^{(1)} + b^{(2)}\big), \qquad \hat{y} = a^{(2)}$$

What it produces

one prediction per row — a probability, a value, a score. Nothing is learned here; the weights are taken as given.

What it costs

a few matrix multiplications per row. This is why inference is cheap and training is not.

Forward propagation is also the diagnostic: if the architecture is right and the output is still poor, the problem is in the weights or in the data — not in the arithmetic.

Backpropagation and the Weight Update

Training compares the prediction with the recorded answer, then pushes the error backwards to assign blame to each weight:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

η is the learning rate; $\partial L/\partial w$ says how much the loss changes if that weight moves. Repeat over batches until the validation loss stops improving.

The four moving parts

- **loss** — what "wrong" means;
- **gradient** — local blame;
- **learning rate** — step size;
- **epochs / early stopping** — when to stop.

Deep networks are trained this way too; "deep" just means more layers to propagate through, which is why vanishing gradients, batch normalisation and residual connections exist.

Source: The update rule is the one shown in the module reading: new weight = old weight − learning rate × derivative of total error.

Deep Learning: What Depth Buys

Representation learning

Early layers detect simple patterns; later layers compose them. On images: edges → textures → parts → objects. You supply raw input, not engineered features.

Where it wins in marketing

creative and image classification, text intent and sentiment, speech, cross-modal search — tasks where the feature is the hard part.

Where it does not

tabular data with a few hundred rows and tidy columns. There, a well-tuned boosted tree usually matches or beats a network — with far less tuning and much better failure behaviour.

Source: This is the practical consensus in applied ML; the AI Foundation decks cover the architecture side.

Reinforcement Learning: Agent, Action, Reward

The five elements

- **agent** — the decision rule being learned;
- **state** — what it observes;
- **action** — what it can do;
- **reward** — the delayed signal;
- **environment** — the world that responds.

Why marketing uses it

budget allocation across channels, bid shading in auctions, sequential messaging, dynamic pricing — decisions where today's choice changes tomorrow's state.

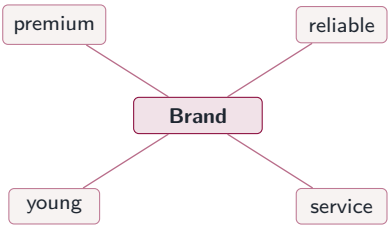
The methodological price: no labelled answers, slow feedback, and a strong temptation to optimise the proxy reward rather than the objective you actually care about.

Source: AlphaGo (2017) made the paradigm famous; business applications are usually contextual bandits rather than games.

Marketing Application: Brand Associations as a Network

The associative network memory model treats brand equity as a graph: a brand node connected to attribute nodes, strengthened by exposure.

- the size of the brand node $\approx$ awareness;
- the number and strength of links $\approx$ the associations the brand owns;
- a placement (a film, an athlete, a partnership) adds a whole constellation of links at once.



The bridge to this lecture: the predictive version of the same idea is a neural network's hidden layer — associations learned from co-occurrence rather than declared by a strategist. Same geometry, different evidence.

Source: Keller's associative network memory model of brand equity, as used in the module's branding reading.

Reading an ML Result: A Checklist

1. **What exactly is the label?** Definition, window, and whether it was available at decision time. A leaked label makes everything downstream meaningless.
2. **How was the split made?** By time and by entity — or by row, which usually leaks.
3. **What is the baseline?** Always report the naive model (predict the mean, predict the majority class) next to yours.
4. **Which metric, at which threshold?** Accuracy on an imbalanced problem is decoration. Report precision/recall at the operating point you will actually use.
5. **Is the improvement stable?** Across folds, across periods, across markets — not only on one lucky split.
6. **Can it be explained enough to be defended?** Feature importance, partial effects, or at minimum the direction of the main drivers.
7. **What happens when the world moves?** Drift, retraining cadence, and the monitoring you would need in production.

Source: This checklist is the basis of the model section of the course project and of the AI-use appendix.

Tools: A Minimal Python Workflow

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

clf = RandomForestClassifier(n_estimators=400, min_samples_leaf=5,
                             random_state=42)
scores = cross_val_score(clf, X_train, y_train, cv=5, scoring="
      roc_auc")
clf.fit(X_train, y_train)
print(scores.mean(), classification_report(y_test, clf.predict(X_test)
      ))
```

- `stratify` keeps the class ratio in both halves;
- `cross_val_score` gives the honest estimate on training data only;
- `random_state` makes the split reproducible — required in the project;
- the alternative stack is R (`caret`, `tidymodels`) or Stata for the econometric side.

Source: scikit-learn user guide; the code ran against the sample data on the course Code page.

Common Pitfalls

Data problems

- **leakage**: a feature computed after the outcome;
- **temporal leakage**: random split on time-series data;
- **class imbalance** ignored, so the model never predicts the rare class;
- **distribution shift** between training and deployment.

Process problems

- tuning on the test set until it flatters the model;
- comparing models fitted on different samples;
- reporting accuracy where cost is asymmetric;
- deploying without a baseline or a monitoring plan.

Most of these are design failures, not modelling failures — and all of them are cheaper to prevent than to discover in a committee meeting.

Summary

1. Machine learning replaces written rules with rules estimated from examples; the three slots to fill are the task, the performance measure and the experience.
2. Generalisation is the only thing that counts: overfitting means memorising noise, underfitting means missing structure, and the bias–variance decomposition explains both.
3. The two levers against overfitting are complexity control (penalisation, pruning, early stopping) and honest sampling (validation, cross-validation, time-based splits).
4. Supervised learners differ mainly in shape and stability; unsupervised methods find structure but need a business criterion to be judged.
5. Neural networks are flexible function approximators; depth buys learned features, and reinforcement learning buys sequential decisions at the cost of slow feedback.
6. A model is only as good as its label, split, baseline, metric and monitoring. Those five choices are where the marks — and the failures — live.

Source: Next: L6 applies these methods to marketing problems — recommendation, attribution, pricing and customer value.

Thank You

Questions, datasets and the code from this lecture are on the course site.